ROS2 Simulation of a Test Tube Washing Pipeline Using a SCARA Robot

Samuel Singh
MAI5303: Applied Robotics
University of Guyana

***Abstract*—Biomedical and life-science laboratories rely on repetitive workflows such as sample transfer, washing, and sorting, which are well-suited for automation due to their precision requirements and contamination sensitivity. This report presents a ROS 2 Jazzy and Gazebo Harmonic simulation of a simplified test-tube washing pipeline using a SCARA manipulator. The implemented system integrates camera-based target detection, pixel-to-world mapping, inverse kinematics, joint-trajectory execution via `ros2_control`, and object manipulation via attachment/detachment interfaces bridged between Gazebo and ROS. While full washing and re-racking stages are not fully implemented, the project validates a complete perception–action pipeline for laboratory pick-and-place automation in simulation.**

***Index Terms*—ROS 2, Gazebo Harmonic, SCARA, laboratory automation, perception, inverse kinematics, ros2_control, simulation**

I. Introduction

A. Problem Description

Biomedical and life-science laboratories commonly execute structured, repetitive tasks such as sample transfer and washing. These tasks demand accuracy and consistency, making them suitable targets for robotic automation. This project simulates a simplified test-tube handling scenario using a SCARA robot in ROS 2 and Gazebo Harmonic. The focus is on detecting a test tube using camera-based perception, mapping image coordinates to an approximate workspace target, and executing a pick-and-place trajectory using joint-space control. Although the complete washing station behavior is not fully implemented, the simulation demonstrates the essential building blocks: perception-driven localization, inverse kinematics, trajectory execution, and interaction with a simulated object.

II. System Architecture Overview

The system is implemented as a ROS 2 node-based perception–action pipeline integrating Gazebo Harmonic simulation, camera-based perception, and joint-space control. Image data generated in simulation is bridged into ROS via `ros_gz_bridge` and processed by a dedicated perception node to localize the target object. The resulting target position is converted into joint commands through inverse kinematics and executed using `ros2_control`. Object manipulation is approximated using attachment and detachment interfaces bridged between Gazebo and ROS.

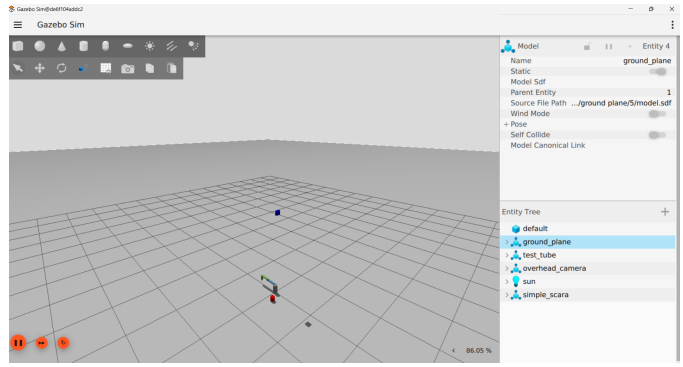

Fig. 1. Gazebo Harmonic laboratory world overview (SCARA robot, test tube, overhead camera).

A. Control and State Publishing

Robot state and actuation are handled using `ros2_control`. The joint state broadcaster publishes `/joint_states`, and `robot_state_publisher` uses the URDF on `/robot_description` to publish the robot state and transforms. A trajectory controller (e.g., `/arm_position_controller`) accepts joint trajectories for execution.

B. Gazebo–ROS Integration

Gazebo Harmonic runs the physics simulation and hosts the robot and world objects. The Gazebo–ROS bridge (`ros_gz_bridge`) provides connectivity for simulated sensor data and manipulation interfaces, including the camera image stream and the attachment/detachment interfaces used to approximate grasping.

III. Robot Design (URDF/Xacro)

A SCARA manipulator is used due to its suitability for planar pick-and-place tasks common in lab automation. The robot model includes two revolute joints for planar motion in the XY plane and a prismatic joint for vertical motion along Z. Link visuals and collisions are defined in the URDF, and inertial properties are provided to improve simulation stability. The same URDF is used by `robot_state_publisher` and `ros2_control` to ensure consistent kinematics and joint naming across the system.

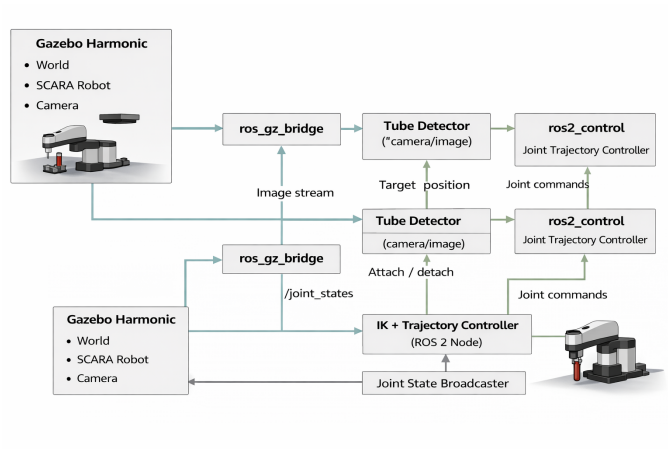

Fig. 2. High-level architecture: Gazebo world and camera provide observations; perception produces a target; inverse kinematics and `ros2_control` generate and execute joint trajectories; attachment/detachment is bridged between Gazebo and ROS.

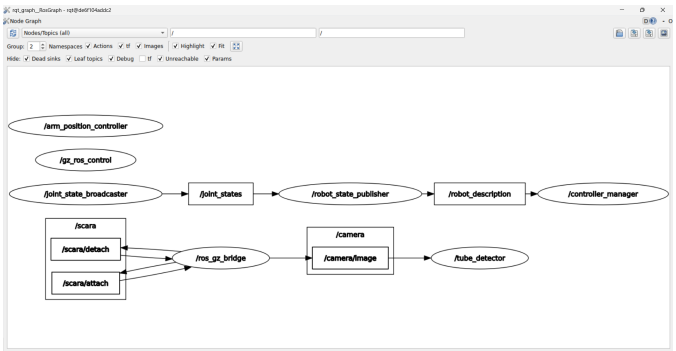

Fig. 3. ROS 2 node graph generated using `rqt_graph`, showing camera image bridging, target detection, `ros2_control` components, and Gazebo attachment interfaces bridged via `ros_gz_bridge`.

IV. Control & Task Logic

The task logic is implemented as a ROS 2 node that executes a single perception-driven pick-and-place cycle. The overall sequence is: detect, hover, attach, transport, and detach.

A. Perception

An overhead camera simulated in Gazebo Harmonic publishes image data that is bridged into ROS on the `/camera/image` topic. A perception node (`/tube_detector`) subscribes to this stream and performs color-based segmentation and contour analysis to identify the test tube. The centroid of the detected object is used as the input to the control stage.

B. Pixel-to-World Mapping

Image coordinates are mapped to approximate workspace coordinates using a calibrated linear mapping. Let (u, v) be the image centroid and (u_0, v_0) the image center. The mapping is:

$$x = S_x(v - v_0), \quad y = S_y(u - u_0) \tag{1}$$

where S_x and S_y are empirically determined scale factors.

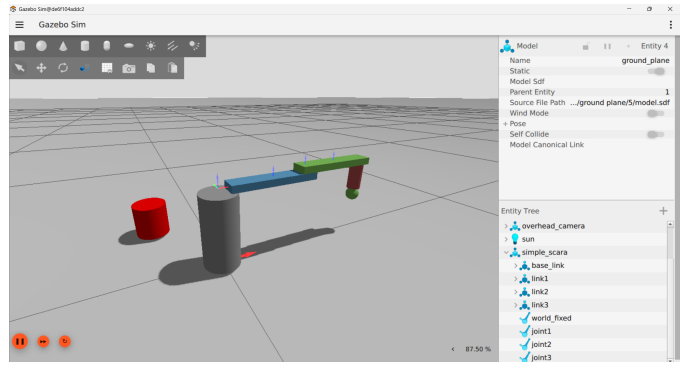

Fig. 4. Gazebo Harmonic visualization of the SCARA model showing link coordinate frames and joint axes.

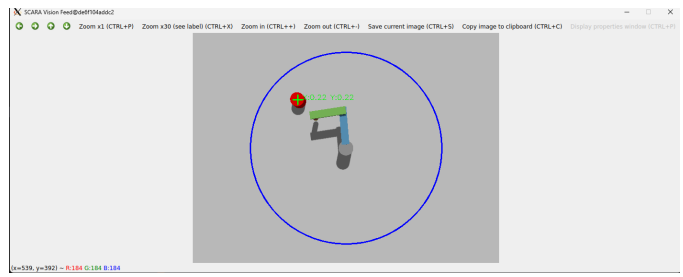

Fig. 5. Perception output showing target segmentation and centroid localization.

C. Inverse Kinematics

A closed-form inverse kinematics solution computes joint angles from the target (x, y), with workspace validation to avoid unreachable targets.

D. Trajectory Execution

Joint-space motion is executed by publishing `trajectory_msgs/JointTrajectory` commands to the trajectory controller. The controller moves to a hover pose, descends in Z, lifts, and translates to a predefined destination pose for placement.

E. Attachment/Detachment

Object manipulation is approximated by creating and removing a fixed joint between the end-effector and the object. Attachment and detachment interfaces are bridged between Gazebo and ROS and are invoked as part of the task sequence.

V. Challenges & Limitations

A. Gripper Plugin Support

Vacuum and magnetic gripper plugins were not reliable in the selected Gazebo Harmonic configuration. A fixed-joint attachment method was used to approximate grasping behavior.

B. Partial Pipeline Completion

The implementation demonstrates a single pick-and-place cycle; extended washing behavior and re-racking are not fully implemented.

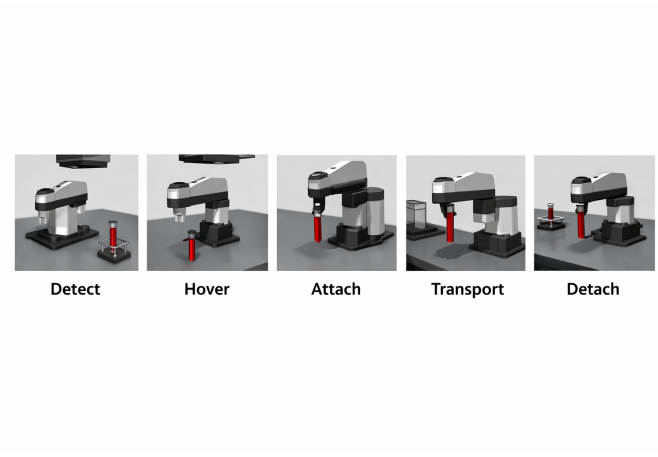

Fig. 6. Task execution timeline: Detect → Hover → Attach → Transport → Detach.

C. Calibration Sensitivity

The pixel-to-world mapping relies on empirical scale factors and is sensitive to camera placement, resolution, and perspective effects.

D. Sequencing Robustness

Task sequencing is implemented procedurally. A formal state machine would improve clarity, recovery from errors, and extensibility to multi-object workflows.

VI. Suggested Future Improvements

Future work can improve completeness and robustness:

- Implement a finite state machine for task sequencing (Idle, Pick, Wash, Place).
- Add a timed washing station and complete a full pipeline loop.
- Improve coordinate estimation via camera calibration and TF-based transformations.
- Add grasp validation (e.g., object pose verification after attachment and before transport).
- Incorporate trajectory smoothing and safety checks for more realistic motion profiles.

Appendix A
Software and Middleware Documentation

References

[1] Open Robotics, "ROS 2 Documentation (Jazzy Jalisco)," 2025. [Online]. Available: https://docs.ros.org/en/jazzy/
[2] Open Robotics, "Gazebo Harmonic Documentation," 2025. [Online]. Available: https://gazebosim.org/docs/harmonic
[3] Open Robotics, "ros2_control: A Framework for Robot Control," 2025. [Online]. Available: https://control.ros.org/
[4] Open Robotics, "Joint Trajectory Controller," 2025. [Online]. Available: https://control.ros.org/master/doc/ros2_controllers/joint_trajectory_controller/doc/userdoc.html
[5] Open Robotics, "robot_state_publisher," 2025. [Online]. Available: https://docs.ros.org/en/jazzy/p/robot_state_publisher/
[6] Open Robotics, "Joint State Broadcaster," 2025. [Online]. Available: https://control.ros.org/master/doc/ros2_controllers/joint_state_broadcaster/doc/userdoc.html
[7] Open Robotics, "ros_gz_bridge: ROS–Gazebo Communication," 2025. [Online]. Available: https://github.com/gazebosim/ros_gz
[8] Open Robotics, "Gazebo Sensors: Camera," 2025. [Online]. Available: https://gazebosim.org/docs/harmonic/sensors
[9] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000. [Online]. Available: https://docs.opencv.org/
[10] Open Robotics, "rqt_graph: ROS Computation Graph Visualization," 2025. [Online]. Available: https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Nodes.html

Appendix B
Additional Screenshots

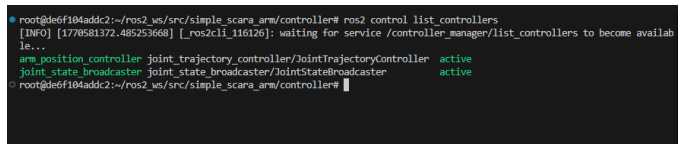

Fig. 7. Controller configuration at runtime showing active `ros2_control` controllers.